



Big Data(BCS061/BCDS-601/KOE-097)



Unit – 5 Hadoop Ecosystem Frameworks , Pig, Hive , HBase

V	Hadoop Eco System Frameworks: Applications on Big Data using Pig, Hive and HBase Pig - Introduction to PIG, Execution Modes of Pig, Comparison of Pig with Databases, Grunt, Pig Latin, User Defined Functions, Data Processing operators,
	Hive - Apache Hive architecture and installation, Hive shell, Hive services, Hive metastore, comparison with traditional databases, HiveQL, tables, querying data and user defined functions, sorting and aggregating, Map Reduce scripts, joins & subqueries. HBase – Hbase concepts, clients, example, Hbase vs RDBMS, advanced usage, schema design, advance indexing, Zookeeper – how it helps in monitoring a cluster, how to build applications with Zookeeper. IBM Big Data strategy, introduction to Infosphere, BigInsights and Big Sheets, introduction to Big SQL.



Edushine Classes



Download Notes : <https://rzp.io/rzp/JV7zlavG>



Big Data(BCS061/BCDS-601)



🐷 What is Pig? (in Hadoop)

Pig is a data flow language used to analyze big data in Hadoop.

It uses a simple language called **Pig Latin**, which is easier than writing Java MapReduce code.

❖ Why Use Pig?

- It helps process **huge data sets**.
- It reduces **coding time** (just like SQL is easier than full programming).
- It converts your code into **MapReduce jobs** automatically.

⚙️ Execution Modes of Pig :

Pig can run in 2 Modes –

Mode	Meaning
Local Mode	Runs on your single computer (good for small data/testing)
<u>MapReduce</u> Mode	Runs on a <u>Hadoop</u> cluster (used for large data processing)



Big Data(BCS061/BCDS-601)



→ You can choose the mode using the command:

`pig -x local` // for local mode

`pig -x mapreduce` // for Hadoop cluster

★ Features of Pig

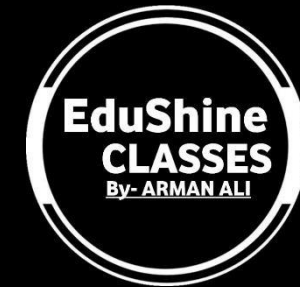
- i. **Easy to Learn** – Uses **Pig Latin**, similar to SQL.
- ii. **Handles Big Data** – Good for analyzing **huge datasets**.
- iii. **Extensible** – You can write your **own functions** (called UDFs).
- iv. **Automatically Converts to MapReduce** – No need to write complex code.
- v. **Supports Joins, Filters, Grouping** – Like SQL operations.
- vi. **Error Handling** – Provides good debugging and error messages.

Pig is a tool to process big data using **Pig Latin**.

It runs in **local** or **MapReduce mode** and makes data handling **easy and fast** in Hadoop.



Big Data(BCS061/BCDS-601)



Pig Latin vs SQL(Database) :

Feature	Pig Latin	SQL
Type	Scripting language	Query language
Execution	Step-by-step (procedural)	One-shot (declarative)
Used In	Hadoop (Pig)	Databases like MySQL, Oracle
Data Flow	You write how data flows (line by line)	You say what result you want
Ease of Learning	Easier for programmers	Easier for database users
Joins, Filters	Supports all	Supports all
Customization	You can write custom functions (UDFs)	Limited customization
Execution Engine	Converts to MapReduce jobs	Uses traditional database engine



Big Data(BCS061/BCDS-601)



🐷💻 What is Grunt in Pig?(Short Note)

- Grunt is the command-line interface (CLI) of Pig.
- It's like a place where you type Pig commands and run them step by step.

✓ What You Can Do in Grunt:

- Write and run Pig Latin commands
- Load, filter, join, and process data
- See outputs and debug easily

💻 Looks like:

When you open Pig in terminal using `pig` command, you will see:

```
shell
```

[Copy](#) [Edit](#)

```
grunt>
```

This is where you type your Pig Latin code.



Big Data(BCS061/BCDS-601)



❖ Syntax and Semantics of Pig Latin :

✓ Syntax of Pig Latin

Pig Latin is a **data flow language**. Its syntax defines **how we write statements** to process data step by step.

It includes commands like:

1.LOAD – To load data from HDFS

```
data = LOAD 'file.txt' USING PigStorage(',') AS (name:chararray, age:int);
```

2.FILTER – To select rows based on condition

```
adults = FILTER data BY age >= 18;
```

3.FOREACH...GENERATE – To select specific columns

```
names = FOREACH adults GENERATE name;
```

4.GROUP – To group records

```
grouped = GROUP data BY age;
```

5.JOIN – To combine two datasets

```
joined = JOIN A BY id, B BY id;
```




Big Data(BCS061/BCDS-601)



6.STORE/DUMP – To save or display the result

DUMP names;

STORE names INTO 'output';

✓ Semantics of Pig Latin

Semantics means the **meaning of the Pig Latin statements**. Each line is a **step in the data flow** and describes how data moves and is processed.

Example :

```
data = LOAD 'students.csv' AS (name, marks);
```

```
passed = FILTER data BY marks >= 33;
```

```
DUMP passed;
```

Meaning:

- Load student data
- Select only those who passed
- Show the result on screen

Pig Latin has a **simple syntax** and **clear semantics**, making it easy to process large data in Hadoop. It supports **step-by-step data flow**, similar to SQL but more flexible for big data.



Big Data(BCS061/BCDS-601)



✓ What is a UDF in Pig?

A **User Defined Function (UDF)** in Pig is a **custom function** created by the user to perform operations that are **not available in built-in functions**.

Pig has many built-in functions, but if you need something special (like custom string or math logic), you can **create your own**.

✓ Language Used:

- ✓ UDFs are usually written in **Java**
- ✓ Can also be written in **Python, Ruby, or JavaScript**

✓ Example Use:

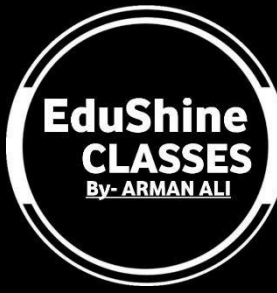
Let's say you want to convert names to uppercase but there's no built-in function:
You can write a UDF like ToUpper() and use it in Pig like:

Example :

```
REGISTER myudfs.jar;  
data = LOAD 'file.txt' AS (name:chararray);  
upper_names = FOREACH data GENERATE ToUpper(name);
```




Big Data(BCS061/BCDS-601)



❖ Data Processing Operators in Pig

Pig Latin provides several **data processing operators** that help in analyzing and transforming large datasets efficiently. These operators allow step-by-step data processing similar to SQL but are more suitable for parallel processing in Hadoop.

◆ 1. LOAD

Used to load data from a file or HDFS into a relation.

◆ 2. FILTER

Used to select records that meet a specific condition.

◆ 3. FOREACH...GENERATE

Used to perform operations on each record and generate new output.

◆ 4. GROUP

Used to group records based on the value of a specific field.

◆ 5. JOIN

Used to join two or more relations based on a common key.



Big Data(BCS061/BCDS-601)



◆ 6. ORDER

Used to sort the data based on one or more fields.

◆ 7. DISTINCT

Used to remove duplicate records from a dataset.

◆ 8. LIMIT

Used to return a specified number of rows.

◆ 9. DUMP

Used to display the result on the console.

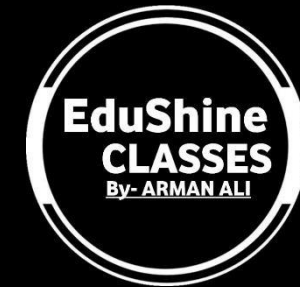
◆ 10. STORE

Used to save the result into a file or directory in HDFS.

These operators are essential for performing tasks like filtering, grouping, joining, and storing data in big data applications using Pig.



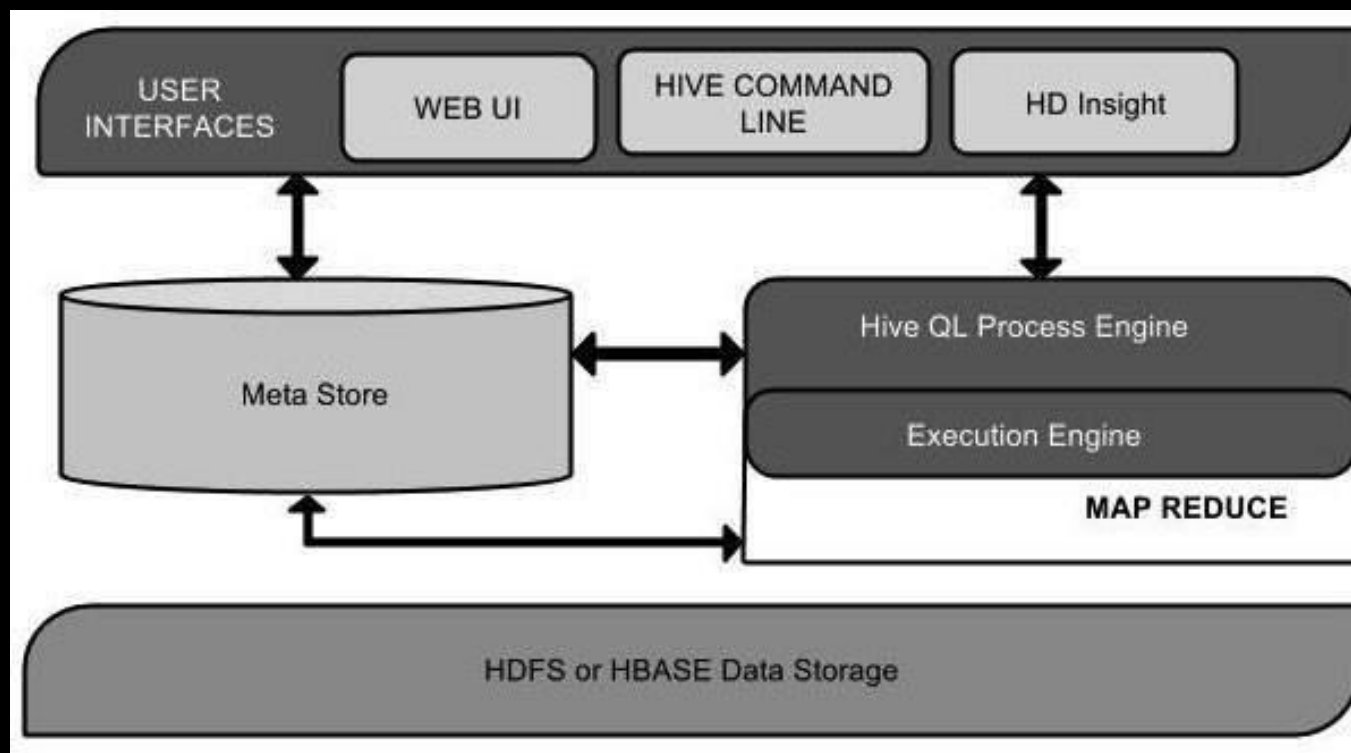
Big Data(BCS061/BCDS-601)

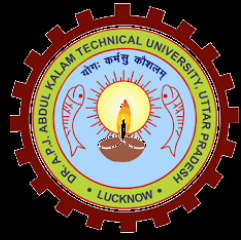


❖ Apache Hive and Its Architecture

◆ What is Hive?

Hive is a data warehouse tool built on top of Hadoop. It helps in reading, writing, and managing large datasets using **HiveQL** (a SQL-like language). It converts HiveQL queries into **MapReduce jobs** for processing.





Big Data(BCS061/BCDS-601)



Architecture of Hive:

1. User Interfaces:

Used to interact with Hive.

Examples:

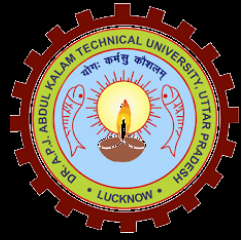
- Web UI
- Hive Command Line
- HDInsight

2. Meta Store:

- Stores **metadata** (info about tables, columns, data types).
- Helps Hive know **where and how** the data is stored in HDFS.

3. HiveQL Process Engine:

- Receives queries written in HiveQL.
- Checks the syntax and passes the query to the execution engine.



Big Data(BCS061/BCDS-601)



4. Execution Engine:

- Converts queries into **MapReduce jobs**.
- Executes them on the Hadoop cluster.

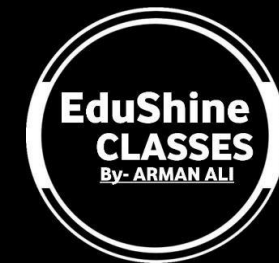
5. HDFS or HBase Storage:

- Hive stores actual data in **HDFS or HBase**.
- It just processes queries over this stored data.

Hive lets you run SQL-like queries on big data stored in HDFS. It uses components like Metastore, HiveQL engine, and Execution engine to turn your queries into results.

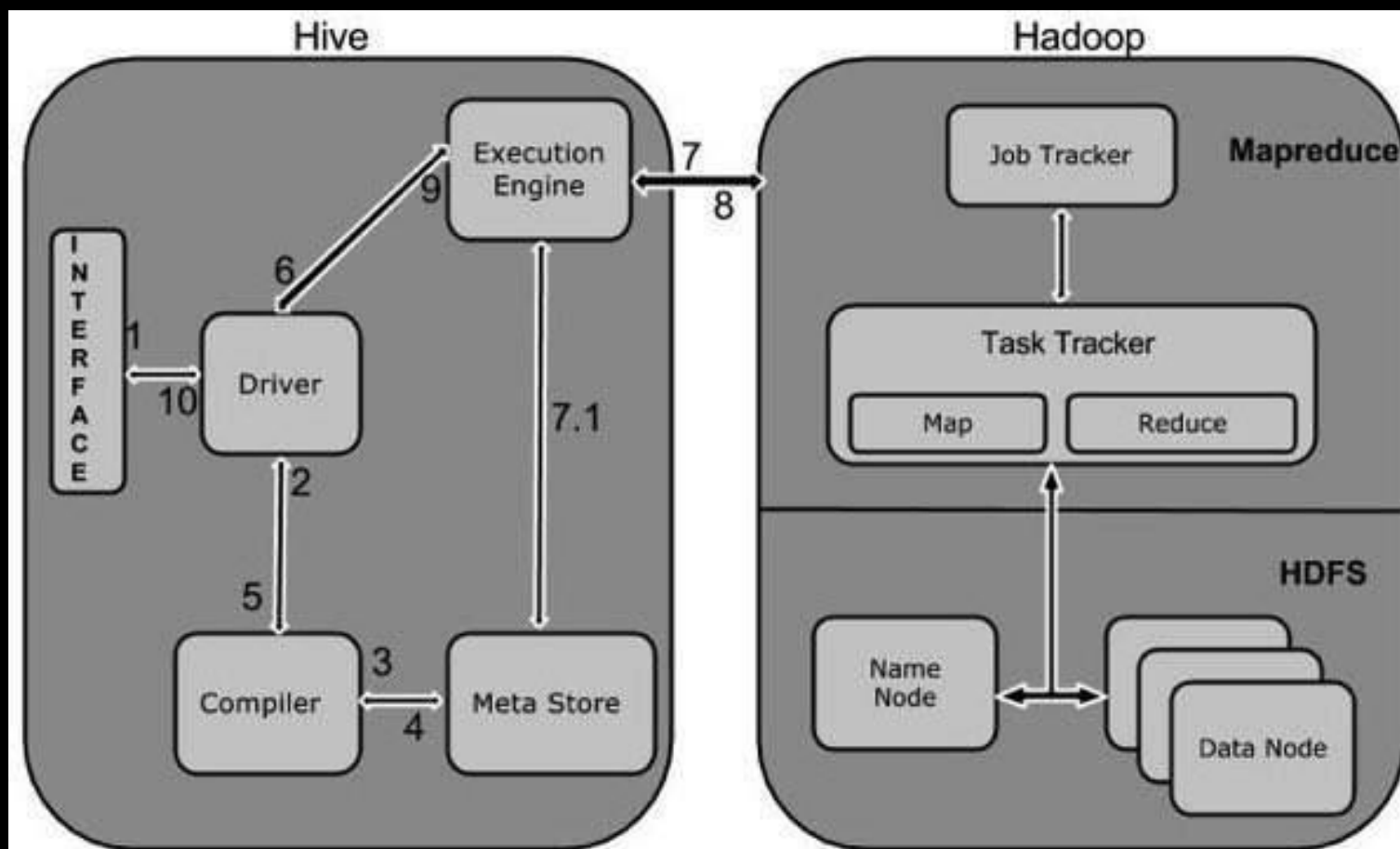


Big Data(BCS061/BCDS-601)



Working of Hive with Hadoop (Step-by-Step)

When a user runs a HiveQL query, this is what happens:





Big Data(BCS061/BCDS-601)



◆ 1. Interface (Step 1 & 10):

The user writes the query using Hive Command Line, Web UI, or other interfaces.

◆ 2. Driver (Steps 2, 6, 9):

The **driver** receives the query and manages the full process:

- Sends the query to the **compiler**
- Monitors the execution
- Returns results to the user

◆ 3. Compiler (Steps 3 & 5):

The **compiler** checks the query for errors and converts it into a logical plan.

It also asks the **Metastore** for table info.

◆ 4. Metastore (Step 4):

Stores **metadata** (data about data), like table names, columns, data types, location in HDFS.

◆ 5. Execution Engine (Steps 7, 7.1, 8):

The query is passed to the **Execution Engine**, which converts it into **MapReduce jobs**.



Big Data(BCS061/BCDS-601)



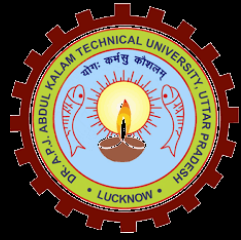
◆ 6. Hadoop Framework (MapReduce + HDFS):

- MapReduce processes the data
- HDFS provides the data from DataNodes
- Once processed, results are sent back to the Hive Execution Engine

◆ 7. Final Result (Step 9 & 10):

The result is collected by the **Driver** and shown to the user.

Hive converts your SQL-like query into **MapReduce jobs**, runs them using Hadoop, gets the results from HDFS, and gives you the answer — just like a smart translator between SQL and big data.



Big Data(BCS061/BCDS-601)



■ Short Note: Apache Hive Installation :

1.Install Java and Hadoop

- Make sure Java and Hadoop are installed and working properly.
- Set environment variables for both.

2.Download Hive

- Go to the official Hive website and download the Hive software.
- Extract the files and place them in a folder like /usr/local/hive.

3.Set Environment Variables

- Add Hive path to the system using .bashrc or .bash_profile.

4.Create Directories in HDFS

- Make folders /tmp and /user/hive/warehouse in HDFS.
- Give permission using Hadoop commands.

5.Initialize Metastore

- Use Derby database (default) and run command to initialize the schema:
schematool -initSchema -dbType derby



Big Data(BCS061/BCDS-601)



6.Start Hive

- Type hive in terminal to open Hive shell and start writing HiveQL queries.

✓ Hive Shell :

Hive Shell is a command-line tool where we write and run Hive queries.

- It looks like a terminal screen where we type **HiveQL** commands.
- It is used to **create tables, load data, and run queries** on big data stored in HDFS.

📝 Example:

You open Hive shell by typing hive in the terminal. Then you can write:

```
SELECT * FROM student;
```

✓ Hive Services :

Hive has several services that help it work smoothly. Main services are:

1. Driver

Manages query execution and keeps track of its progress.



Big Data(BCS061/BCDS-601)



2. Compiler

Checks your Hive query and converts it into a MapReduce job.

3. Metastore

Stores information (metadata) about Hive tables like names, columns, types, etc.

4. Execution Engine

Runs the query and fetches the result using MapReduce.

✓ What is Hive Metastore?

- **Hive Metastore** is like a **library catalog** for Hive.
 - It stores all the **information about Hive tables**—like their names, columns, data types, where data is stored, etc.
- ✚ Think of it as a **database about your data**.



Big Data(BCS061/BCDS-601)



Example:

If you create a table in Hive:

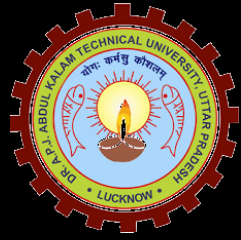
sql

```
CREATE TABLE student (id INT, name STRING);
```

Hive Metastore saves details like:

- Table name: `student`
- Columns: `id`, `name`
- Location of data in HDFS

Hive Metastore is a service that stores metadata about Hive tables, columns, data types, and HDFS locations. It helps Hive know how and where the data is stored.

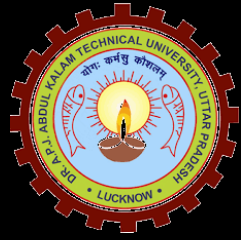


Big Data(BCS061/BCDS-601)



✓ Comparison: Hive vs Traditional Database

Feature	Hive (on Hadoop)	Traditional Database (RDBMS)
Language Used	HiveQL (similar to SQL)	SQL
Data Storage	Stores data in HDFS (<u>Hadoop Distributed File System</u>)	Stores data in local disks or servers
Processing Type	Batch processing (not real-time)	Real-time processing
Data Size	Best for huge data (terabytes/petabytes)	Handles limited/medium-sized data
Speed	Slower (because of MapReduce jobs)	Faster (especially for small data)
Schema Type	Schema on read (applies schema when reading)	Schema on write (strict structure at save)
Use Case	Big data analytics, reporting, summarizing	Online transactions, banking, ERP, etc.
Transactions	Not good for transactional processing	Supports transactions (ACID properties)



Big Data(BCS061/BCDS-601)



✓ 1. What is HiveQL?

HiveQL (Hive Query Language) is a **SQL-like language** used to interact with Hive. It helps to **create tables, insert data, and run queries** on large datasets stored in **HDFS**.

✦ Example:

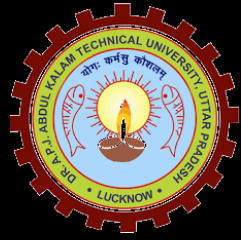
```
SELECT name FROM students WHERE marks > 80;
```

✓ 2. What is a Hive Table?

A **Hive table** is like a **virtual table** where data is stored in HDFS. It has **rows and columns** just like in SQL.

✦ Types:

- i. **Managed Table:** Hive manages both data and metadata.
- ii. **External Table:** Hive manages only metadata. Data remains outside.



Big Data(BCS061/BCDS-601)



✓ 3. What is Partition in Hive?

Partition means dividing a table into **smaller parts** based on column values. Helps in **faster query** performance by scanning only required parts.

✦ Example:

Partition a sales table by year:

➤ PARTITIONED BY (year INT)

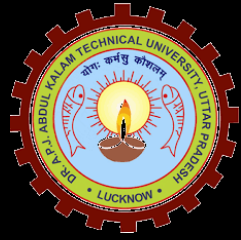
✓ 4. What is Bucketing in Hive?

Bucketing further divides data inside a partition into **equal-sized files** (buckets) based on the **hash function**.

Helps in **faster joins and sampling**.

✦ Example:

➤ CLUSTERED BY (student_id) INTO 4 BUCKETS;



Big Data(BCS061/BCDS-601)



✓ 5. Storage Formats in Hive

Hive supports multiple file formats for storing data:

Format	Description
<u>TextFile</u>	Default format (simple text)
ORC	Compressed, good for Hive performance
Parquet	Columnar format, fast and efficient
Sequence	Splittable binary format (Hadoop native)
Avro	Best for row-based and dynamic schemas

✓ 6. Sorting in Hive

- **Sorting** means arranging data in **ascending or descending order**.
- Done using ORDER BY or SORT BY.
- ✦ Example: `SELECT * FROM student ORDER BY marks DESC;`



Big Data(BCS061/BCDS-601)



✓ 7. Aggregating in Hive

Aggregation means using functions like **COUNT, SUM, AVG, MAX, MIN** to summarize data.

✦ Example: `SELECT AVG(marks) FROM student;`

✓ 8. Joins in Hive

Joins are used to **combine rows from two or more tables** based on a related column.

✦ **Types:**

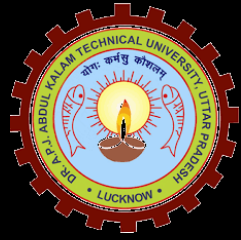
- ✓ **INNER JOIN** – returns matching rows
- ✓ **LEFT OUTER JOIN** – returns all from left + match from right
- ✓ **RIGHT OUTER JOIN** – returns all from right + match from left
- ✓ **FULL OUTER JOIN** – all rows from both tables

Example :

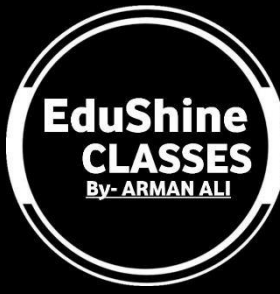
```
SELECT s.name, m.marks
```

```
FROM students s
```

```
JOIN marks m ON s.id = m.student_id;
```



Big Data(BCS061/BCDS-601)



✓ 9. Subqueries in Hive

A subquery is a query inside another query.
It helps in filtering, grouping, or complex logic.

★ Example:

```
SELECT name FROM student  
WHERE marks > (SELECT AVG(marks) FROM student);
```



Big Data(BCS061/BCDS-601)



✓ What is HBase?

- **HBase is a NoSQL database that runs on top of Hadoop.**
- **It is used to store and manage very large data (billions of rows) in a table format, just like an Excel sheet — but distributed across many machines.**
- **It works well for real-time read and write of big data.**
- ✚ **Think of it as a giant Excel sheet spread across many computers!**

✨ Features of Hbase :

Feature	Description (in easy words)
1. NoSQL Database	Not like SQL; it stores data in key-value pairs (column-oriented).
2. Real-Time Access	Supports fast read and write of big data (unlike Hive, which is batch).
3. Scalability	Can grow easily — store billions of records across many machines.
4. Runs on HDFS	Uses Hadoop's HDFS to store data safely and reliably.
5. Column-Oriented	Data is stored in columns, not rows — helps with fast retrieval.
6. Fault-Tolerant	If one machine fails, data is still safe because of replication.
7. Supports Versioning	Keeps multiple versions of data for backup/history.
8. Integration with MapReduce	Can process <u>HBase</u> data using <u>Hadoop MapReduce</u> jobs.



Big Data(BCS061/BCDS-601)

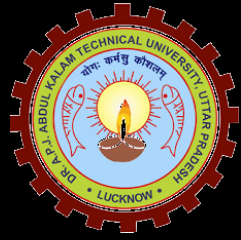


✓ HBase Data Model :

HBase stores data in **tables**, just like SQL — but the structure is **different and more flexible**.

📁 Basic Structure of HBase:

Row Key	Column Family:Column	Timestamp	Value
101	<u>student:name</u>	001	Arman
101	student:marks	001	95
102	student:name	001	Ali
102	student:marks	001	88



Big Data(BCS061/BCDS-601)



❖ HBase Data Model Components :

Component	Explanation (Easy Words)
1. Table	HBase data is stored in tables (like Excel sheets).
2. Row Key	Unique ID for each row (like a roll number). Used to find data quickly .
3. Column Family	Group of related columns. Defined during table creation. (e.g., "student")
4. Column Qualifier	The actual column name inside the family (e.g., "name", "marks").
5. Cell	The combination of row key + column + value.
6. Timestamp	Each cell stores multiple versions of data, each with a timestamp .



Big Data(BCS061/BCDS-601)



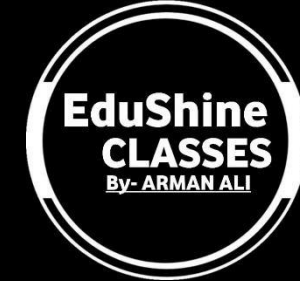
✓ Client Options for Interacting with HBase Cluster

There are many ways to interact with an HBase cluster:

1. **HBase Shell** – This is a command-line tool that lets us run commands to create tables, insert data, read data, and manage the database easily.
2. **Java API** – Developers can use Java programming to connect with HBase and perform read/write operations in their programs.
3. **REST API** – HBase can be accessed using web URLs, which is helpful for web applications and services.
4. **Thrift API** – It allows other languages like Python, PHP, and C++ to connect with HBase.
5. **MapReduce** – Hadoop's MapReduce can be used to process data stored in HBase in large batches.
6. **Hive Integration** – Hive can be used to write SQL-like queries (HiveQL) on HBase



Big Data(BCS061/BCDS-601)



❖ Difference between HBase and RDBMS :

Point	HBase (NoSQL)	RDBMS (SQL)
Type	<u>NoSQL</u> database	Relational database
Storage Model	Column-oriented	Row-oriented
Schema	Schema-less (flexible columns)	Fixed schema (predefined columns)
Scalability	Highly scalable (for big data)	Limited scalability
Query Language	No standard SQL (uses APIs or HiveQL)	Uses SQL
Data Volume	Works well with very large data	Best for small to medium data
Transactions	Not fully ACID-compliant	Fully ACID-compliant
Speed	Fast for large-scale read/write	Fast for small transactions
Storage	Uses HDFS (distributed file system)	Uses local/distributed databases
Use Cases	Real-time apps, logs, time-series, <u>IoT</u> data	Banking, ERP, e-commerce



Big Data(BCS061/BCDS-601)



✓ Schema Design in Hbase :

In HBase, designing the schema means deciding **how to organize your data in tables**. But it's very different from SQL databases.

- HBase is **schema-less** for columns — you only need to define **column families**, not individual columns.
- Each row is identified by a **Row Key** — it should be **unique** and **well-designed** (like a roll number or user ID).
- Column families group related columns (like **student:name, student:marks**).
- It's important to group data that is usually accessed together into the **same column family**.
- Avoid putting too many column families because each one is stored separately, which slows down performance.



Download Notes : <https://rzp.io/rzp/JV7zlavG>



Big Data(BCS061/BCDS-601)



✓ What is Indexing in HBase?

In HBase, data is stored and searched **based on Row Keys only**.

That means:

If you know the **Row Key**, data retrieval is **very fast**.

But if you want to search by some **other column**, like "name" or "city", it becomes slow — because HBase doesn't create indexes on those columns by default.

✓ What is Advanced Indexing?

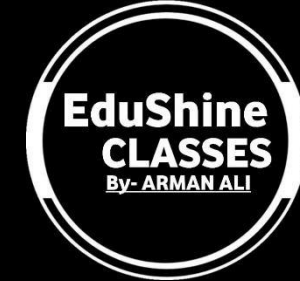
Advanced Indexing means creating a **secondary index** (extra structure) to make searching faster by **non-key columns**.

This helps you search HBase tables like SQL-style queries:

- Search by **name**, **email**, or **age**, not just Row Key.



Big Data(BCS061/BCDS-601)



✓ Example :

Suppose you have an HBase table Student:

Row Key	Name	Age	City
1001	Arman	20	Lucknow
1002	Ravi	21	Kanpur
1003	Priya	22	Delhi

if you want:

"Find student whose **Name = Priya**"

→ □ This is slow because HBase will check each row one by one (called full scan).

We can create a **Secondary Index Table**:

Index Row Key (Name)	Student Row Key
Arman	1001
Ravi	1002
<u>Priya</u>	1003



Big Data(BCS061/BCDS-601)



Now:

First, you search in the index table using "Priya" → it gives you 1003.

Then, go to the main table with 1003 → get full student data.

✓ Faster than full table scan.

✓ Short Note on ZooKeeper and Its Role in Monitoring a Cluster

ZooKeeper is a tool used in Hadoop and HBase systems to **manage and coordinate** different machines (nodes) in a cluster.

It helps in:

- **Tracking node status:** ZooKeeper keeps an eye on which servers are **active** and which are **down**.
- **Leader election:** If the main/master server fails, ZooKeeper helps to **choose a new leader** automatically.
- **Communication:** It helps all nodes in the cluster **talk to each other smoothly**.
- **Fail recovery:** When a server fails, ZooKeeper informs the system so it can **recover quickly**.



Big Data(BCS061/BCDS-601)



- ZooKeeper makes sure that the **cluster runs smoothly**, with **less downtime** and **better coordination**.

✓ **IBM Big Data Strategy :**

IBM's Big Data strategy focuses on **helping businesses** use their data in a smart way to make **better decisions, faster**.

IBM believes that **Big Data is not just about collecting a lot of data**, but about using that data to get useful insights.

✓ **Key Points of IBM's Big Data Strategy:**

1. Volume, Variety, Velocity:

IBM handles all types of data – big in size, different in format (text, video, etc.), and coming at high speed.

2. Unified Platform:

IBM provides a **complete platform** where you can **store, manage, analyze, and visualize** your data in one place.



Big Data(BCS061/BCDS-601)



3. Infosphere BigInsights:

IBM offers this tool to process and analyze Big Data using **Hadoop** technology.

4. Big SQL:

You can use **SQL queries** to analyze big data easily, even if it's stored in Hadoop.

5. Security and Governance:

IBM ensures that data is **safe, secure, and managed properly**, with proper rules.

6. Integration with AI and Cloud:

IBM connects Big Data with **AI (Watson)** and **Cloud** to provide **real-time intelligence** and smart decisions.



Big Data(BCS061/BCDS-601)



✓ 1. InfoSphere (by IBM)

InfoSphere is a set of IBM tools that helps in:

- **Collecting, managing, and analyzing** big data.
- It makes sure data is **clean, organized, and ready** to be used in analytics.
- It supports **data integration, data quality, and data governance**.

✈ In Easy Words:

InfoSphere is IBM's tool to manage big data properly so companies can trust and use their data easily.

✓ 2. BigInsights

BigInsights is IBM's platform for working with **Big Data using Hadoop**.

- It is built on **Apache Hadoop** but has **extra features** like better security, analytics, and a user-friendly interface.
- Helps to process **large data** and get **useful results**.
- Includes tools for developers, data scientists, and business users.



Big Data(BCS061/BCDS-601)



✦ In Easy Words:

BigInsights is IBM's software that adds more power and features to Hadoop for better big data processing.

✓ 3. BigSheets

BigSheets is a tool in BigInsights that looks like **Excel** but works on **Big Data**.

- It allows users to **analyze large datasets without coding**.
- You can **filter, sort, group, and visualize** big data using an easy spreadsheet-style interface.
- Great for **business users** who don't know programming.

✦ In Easy Words:

BigSheets is like Excel for Big Data. It helps non-technical people explore and analyze big data in a simple way.



Big Data(BCS061/BCDS-601)



✓ What is BigSQL?

BigSQL is a tool by IBM that lets you use **SQL queries** to work with **Big Data** stored in Hadoop.

- Just like we use **SQL** for normal databases (like MySQL, Oracle),
- With **BigSQL**, we can write **same SQL queries** to read data from **Hadoop (HDFS), Hive, or HBase**.

✈ In Easy Words:

BigSQL helps you use familiar SQL language to work with huge data stored in big data systems like Hadoop.

✓ Key Features of BigSQL

- ✓ Works with **standard SQL**
- ✓ Can access data from **Hive, HDFS, HBase**
- ✓ **Faster** and more **efficient** than using Hive alone
- ✓ Supports **joins, subqueries, sorting, grouping**
- ✓ Provides **security and governance** features



Big Data(BCS061/BCDS-601)



✓ How BigSQL Works?

You write SQL queries

Like:

```
SELECT * FROM customers WHERE city = 'Lucknow';
```

2. ⚙️ **BigSQL takes your SQL and translates** it into commands that Hadoop can understand.
3. 📁 **It fetches data** from different big data sources like HDFS, Hive tables, or HBase.
4. ⚡ **Processes the data** using a powerful engine (faster than normal Hive).
5. 📄 **Returns results** just like a normal SQL database does.



Download Notes : <https://rzp.io/rzp/JV7zlavG>



Big Data(BCS061/BCDS-601)



Thank You....



Download Notes : <https://rzp.io/rzp/JV7zlavG>